Image Creation (Java)

Optimizing Docker Images

The size of a Docker image can significantly impact performance for both developers and organizations:

- **Space**: Large images consume more disk space in registries and production servers.
- **Bandwidth**: Pulling and pushing large images consumes more network bandwidth.
- **Speed**: Larger images take longer to build, push, and deploy.
- **Security**: Bigger images often include unnecessary dependencies, increasing the attack surface.

Choosing the Right Base Image

Example with standard Eclipse Temurin JDK:

```
FROM eclipse-temurin:21
ARG JAR_FILE=target/*.jar
COPY ${JAR_FILE} application.jar
ENTRYPOINT ["java","-jar","/application.jar"]
```

> *Image size: ~800MB*

Slim Alpine-based image:

```
FROM eclipse-temurin:21-jdk-alpine
ARG JAR_FILE=target/*.jar
COPY ${JAR_FILE} application.jar
ENTRYPOINT ["java","-jar","/application.jar"]
```

> *Image size: ~650MB*

Tip: Choosing a smaller base image reduces size and improves build/deploy performance.

Multi-Stage Dockerfiles

`jlink` can create a **custom JRE** with only the modules your app needs. Combined with **multi-stage builds**, you can produce very slim images:

```
# First stage: build a custom JRE
FROM eclipse-temurin:21-jdk-alpine AS jre-builder

RUN $JAVA_HOME/bin/jlink \
         --verbose \
         --add-modules ALL-MODULE-PATH \
         --strip-debug \
         --no-man-pages \
         --no-header-files \
         --compress=2 \
         --output /optimized-jdk-21

# Second stage: package app with the custom JRE
FROM alpine:latest
ENV JAVA_HOME=/opt/jdk/jdk-21
ENV PATH="${JAVA_HOME}/bin:${PATH}"

COPY --from=jre-builder /optimized-jdk-21 $JAVA_HOME

ARG APPLICATION_USER=spring
RUN addgroup --system $APPLICATION_USER && adduser --system $APPLICATION_USER --ingroup $APPLICATION_USER
COPY --chown=$APPLICATION_USER:$APPLICATION_USER target/*.jar /application.jar
USER $APPLICATION_USER

ENTRYPOINT ["java","-jar","/application.jar"]
```

> *Image size: ~300MB*

This approach drastically reduces image size while keeping your application fully functional.

Spring Boot Maven Plugin

Spring Boot supports building Docker images natively using Cloud Native Buildpacks (OCI images).

```
<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
    </plugins>
</build>
```

Build and package the image:

```
mvn clean package -Dmaven.test.skip=true
mvn spring-boot:build-image
```

> *Image size: ~700MB*

This method automatically layers the JAR for faster builds and deploys.

Building an Image with Jib (Maven Plugin)

Jib is an open-source tool by Google that builds Docker images for Java applications **without a Dockerfile**.

```
<build>
    <plugins>
        <plugin>
            <groupId>com.google.cloud.tools</groupId>
            <artifactId>jib-maven-plugin</artifactId>
            <version>3.5.1</version>
        </plugin>
    </plugins>
</build>
```

Build and push to Docker Hub:

```
mvn compile jib:build
```

Build for the local Docker daemon:

```
mvn compile jib:dockerBuild
```

> *Image size: ~500MB*

Tip: Jib automatically optimizes layers for dependencies, application code, and resources.

Resources

- Docker Overview
- Docker Getting Started Guide
- Creating Docker Images with Spring Boot
- Spring Boot Maven Plugin - Layered Jars